

SISTEMAS DE ARMAZENAMENTO DE DADOS PARA AUTOMAÇÃO INDUSTRIAL

**Manual de Administração e Gerenciamento de Banco
de Dados**

Programação para Coleta de Dados em Automação
Prof. William Silva – 2026

**GUILHERME SANTOS
EMILLY SANTOS**

**SÃO PAULO / GRU
2026**

Sumário

Identificação do Documento	3
1. Objetivos.....	3
2. Por Que Gerenciar Dados?	3
2.1 Consequências da Falta de Política	3
3. Política de Retenção de Dados.....	4
3.1 Tabela de Retenção por Tipo de Dado	4
3.2 Locais de Armazenamento (Estratificação)	4
3.3 Autorização para Descarte	5
4. Normas de Auditoria e Conformidade.....	5
4.1 O Que Deve Ser Auditado	5
4.2 Implementação com Triggers MySQL	5
4.3 Conformidade com a LGPD	6
5. Procedimento Operacional Padrão (POP)	6
POP-01 — Arquivamento de Leituras de Sensores.....	6
Objetivo	6
Passo a Passo	6
POP-02 — Anonimização de Clientes Inativos (LGPD)	6
Objetivo	6
Passo a Passo	6
POP-03 — Limpeza de Logs de Acesso	7
Passo a Passo	7
6. Instruções de Trabalho.....	7
7. Matriz de Riscos	12
8. Estrutura do Projeto MySQL	12
8.1 Tabelas Implementadas	12
8.2 Stored Procedures	12
8.3 Eventos Agendados.....	12
8.4 Triggers de Auditoria	13
8.5 Views Operacionais	13
9. Conclusão.....	13

Identificação do Documento

Documento	Manual de Gestão de Banco de Dados – Automação Industrial
Versão	1.0
Data	17/04/2026
Autor	GUILHERME SANTOS – EMILLY SANTOS
Disciplina	Programação para Coleta de Dados em Automação
Status	TESTE 02

1. Objetivos

Este documento estabelece práticas e procedimentos para a administração e gestão de bancos de dados em um contexto de automação industrial, incluindo

- Políticas de retenção de dados e ciclo de vida das informações
- Normas de auditoria e conformidade (LGPD, legislação fiscal)
- Instruções de Trabalho (IT) para operações críticas no banco
- Procedimentos Operacionais Padrão (POP) para gestão recorrente
- Scripts MySQL prontos para implementação imediata

2. Por Que Gerenciar Dados?

Armazenar informações não é a única função de um banco de dados. Uma empresa que armazena tudo indefinidamente enfrenta sérios problemas, como lentidão no banco de dados, aumento dos custos de armazenamento, duplicação de dados e riscos jurídicos. Por outro lado, a exclusão prematura de dados pode comprometer provas jurídicas, histórico financeiro e oportunidades de recuperação operacional.

2.1 Consequências da Falta de Política

- Cada funcionário toma decisões diferentes sobre o que guardar
- Banco de dados cresce sem controle, aumentando custos
- Falta de evidências em auditorias externas
- Vazamento de dados pessoais antigos desnecessariamente retidos
- Multas por violação à LGPD (Lei 13.709/2018)

3. Política de Retenção de Dados

A política de retenção é o conjunto de regras que define o ciclo de vida das informações: o que armazenar, por quanto tempo, onde, quando excluir e quem autoriza o descarte.

3.1 Tabela de Retenção por Tipo de Dado

A tabela abaixo define os prazos e locais de armazenamento para cada tipo de informação do sistema de automação industrial:

Tipo de Dado	Retenção	Onde Armazenar	Base Legal / Motivo
Cadastro de cliente ativo	Enquanto ativo	Tabela principal	Necessidade operacional
Cliente inativo	2 anos	Tabela principal	LGPD Art. 16
Nota Fiscal	5 anos	Tabela principal	Legislação fiscal brasileira
Leituras de sensores (online)	12 meses	Tabela online	Performance e custo
Leituras de sensores (frias)	5 anos	Arquivo frio / cloud	Histórico de processo
Log de acesso	1 ano	Tabela de auditoria	Auditoria e conformidade
Backup diário	30 dias	NAS / Cloud	Recuperação operacional
Currículo de candidato	6 meses	RH – base separada	LGPD – dado pessoal
Dados de teste	Não guardar	Nunca em produção	Qualidade de dados
Dados duplicados	Não guardar	—	Performance e custo

3.2 Locais de Armazenamento (Estratificação)

- Base de produção (servidor principal): dados ativos e dos últimos 12 meses
- Tabela histórica / arquivo frio: dados de 1 a 5 anos, acesso eventual
- NAS / Cloud (backup): cópias de segurança rotacionadas a cada 30 dias
- Servidor separado de logs: registros de auditoria e acesso

3.3 Autorização para Descarte

O descarte de dados é o ponto mais negligenciado nas empresas. Nunca deve ser feito por qualquer funcionário sem aprovação formal. A tabela abaixo define as responsabilidades:

Área	Responsabilidade	Exemplo de Ação
Gestor da Área	Aprovação funcional do descarte	Assina o formulário de descarte
TI / DBA	Execução técnica	Executa a stored procedure de descarte
Jurídico	Avalia risco legal	Confirma prazo legal atingido
Compliance	Valida alinhamento com política	Verifica conformidade com LGPD
Segurança da Info	Garante rastreabilidade	Verifica log antes e após descarte

Todo descarte deve gerar um registro na tabela registro_descarte do banco de dados, garantindo rastreabilidade total.

4. Normas de Auditoria e Conformidade

4.1 O Que Deve Ser Auditado

- Inserções, alterações e exclusões em tabelas críticas (clientes, notas fiscais)
- Acessos negados ao sistema
- Execução de procedures de descarte e arquivamento
- Alterações em dados pessoais (campos de identificação)
- Falhas em rotinas de backup

4.2 Implementação com Triggers MySQL

O script SQL do projeto implementa triggers automáticos que registram alterações críticas em logs_acesso sem intervenção manual:

```
-- Trigger: registra DELETE em notas_fiscais automaticamente
CREATE TRIGGER trg_audit_nf_delete
BEFORE DELETE ON notas_fiscais FOR EACH ROW BEGIN
    INSERT INTO logs_acesso (usuario, acao, tabela_afetada, detalhes)
    VALUES (CURRENT_USER(), 'DELETE', 'notas_fiscais', JSON_OBJECT(...));
END;
```

4.3 Conformidade com a LGPD

A Lei Geral de Proteção de Dados (Lei 13.709/2018) exige que dados pessoais sejam tratados com finalidade definida e eliminados quando não houver mais necessidade legal. O projeto implementa:

- Anonimização automática de clientes inativos há mais de 2 anos
 - Registro formal de todo descarte com autorização documentada
 - Evento agendado mensal para execução da anonimização
 - Histórico anonimizado preservando somente dados de negócio sem identificação pessoal
-

5. Procedimento Operacional Padrão (POP)

POP-01 — Arquivamento de Leituras de Sensores

Periodicidade: Diária (automática via evento agendado) | **Executor:** Sistema / DBA

Objetivo

Mover leituras de sensores com mais de 12 meses da tabela online para arquivo frio, mantendo performance do banco de produção.

Passo a Passo

- Verificar se o evento evt_arquivar_leituras está ativo: SHOW EVENTS;
- O evento executa automaticamente às 02:00 todos os dias
- Confirmar execução: SELECT * FROM registro_descarte ORDER BY descartado_em DESC LIMIT 5;
- Em caso de falha, executar manualmente: CALL sp_arquivar_leituras_antigas(12, 'dba_responsavel', @qtd);
- Verificar quantidade arquivada: SELECT @qtd;

POP-02 — Anonimização de Clientes Inativos (LGPD)

Periodicidade: Mensal (dia 1 às 03:00) | **Executor:** Sistema / Compliance

Objetivo

Remover dados pessoais de clientes inativos há mais de 2 anos, cumprindo LGPD Art. 16, preservando somente estatísticas de negócio anonimizadas.

Passo a Passo

- Gerar relatório de clientes candidatos: SELECT * FROM vw_clientes_para_descarte;
- Obter aprovação formal do Gestor, Jurídico e Compliance
- Executar (ou aguardar evento): CALL sp_anonimizar_clientes_inativos(2, 'compliance_responsavel', @qtd);
- Confirmar histórico anonimizado: SELECT * FROM clientes_historico ORDER BY arquivado_em DESC;
- Verificar registro de descarte: SELECT * FROM registro_descarte WHERE tabela_origem = 'clientes';

POP-03 — Limpeza de Logs de Acesso

Periodicidade: Trimestral | Executor: DBA / Segurança da Informação

Passo a Passo

- Verificar volume atual: `SELECT COUNT(*), MIN(acessado_em), MAX(acessado_em) FROM logs_acesso;`
- Confirmar com Segurança da Informação que não há investigação ativa
- Executar: `CALL sp_limpar_logs_antigos(12, 'dba_responsavel', @qtd);`
- Verificar: `SELECT @qtd; -- quantidade removida`

6. Instruções de Trabalho

```
✓ /* =====  
PROJETO PROFESSOR – BANCO COMPLETO SQL SERVER  
===== */  
  
CREATE DATABASE automacao_industrial;  
GO  
  
USE automacao_industrial;  
GO  
  
/* =====  
TABELA CLIENTES  
===== */  
CREATE TABLE clientes (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    nome VARCHAR(150) NOT NULL,  
    cpf_cnpj VARCHAR(20) NOT NULL UNIQUE,  
    email VARCHAR(150),  
    telefone VARCHAR(20),  
    ativo BIT NOT NULL DEFAULT 1,  
    inativado_em DATE NULL  
);  
GO
```

```

/* =====
TABELA EQUIPAMENTOS
===== */
CREATE TABLE equipamentos (
    id INT IDENTITY(1,1) PRIMARY KEY,
    tag VARCHAR(50) NOT NULL UNIQUE,
    descricao VARCHAR(200) NOT NULL,
    localizacao VARCHAR(100),
    tipo VARCHAR(20) NOT NULL,
    ativo BIT NOT NULL DEFAULT 1,
    instalado_em DATE NOT NULL,
    desativado_em DATE NULL,
    CONSTRAINT chk_tipo_equip
        CHECK (tipo IN ('sensor', 'atuador', 'controlador', 'medidor'))
);
GO

```

```

/* =====
LEITURAS SENSORES
===== */
CREATE TABLE leituras_sensores (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    equipamento_id INT NOT NULL,
    valor DECIMAL(12,4) NOT NULL,
    unidade VARCHAR(20) NOT NULL,
    qualidade VARCHAR(10) NOT NULL DEFAULT 'bom',
    coletado_em DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT fk_leit_equip
        FOREIGN KEY (equipamento_id)
        REFERENCES equipamentos(id),
    CONSTRAINT chk_qualidade
        CHECK (qualidade IN ('bom', 'suspeito', 'ruim'))
);
GO

```

```

/* =====
NOTAS FISCAIS
===== */
CREATE TABLE notas_fiscais (
    id INT IDENTITY(1,1) PRIMARY KEY,
    cliente_id INT NOT NULL,
    numero_nf VARCHAR(20) NOT NULL UNIQUE,
    serie VARCHAR(5),
    valor_total DECIMAL(14,2) NOT NULL,
    emitida_em DATE NOT NULL,
    cancelada_em DATE NULL,
    CONSTRAINT fk_nf_cliente
        FOREIGN KEY (cliente_id)
        REFERENCES clientes(id)
);
GO

```



```
/* =====  
BACKUPS  
===== */  
CREATE TABLE backups (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    tipo VARCHAR(20) NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    tamanho_mb DECIMAL(10,2),  
    caminho_arquivo VARCHAR(500),  
    iniciado_em DATETIME2 NOT NULL,  
    finalizado_em DATETIME2 NULL,  
    criado_em DATETIME2 NOT NULL DEFAULT SYSDATETIME(),  
    CONSTRAINT chk_backup_tipo  
        CHECK (tipo IN ('completo', 'incremental', 'diferencial')),  
    CONSTRAINT chk_backup_status  
        CHECK (status IN ('sucesso', 'falha', 'parcial'))  
);  
GO  
  
/* =====  
LOGS DE ACESSO  
===== */  
CREATE TABLE logs_acesso (  
    id BIGINT IDENTITY(1,1) PRIMARY KEY,  
    usuario VARCHAR(100) NOT NULL,  
    acao VARCHAR(200) NOT NULL,  
    tabela_afetada VARCHAR(100),  
    registro_id INT NULL,  
    ip_origem VARCHAR(45),  
    resultado VARCHAR(10) NOT NULL DEFAULT 'sucesso',  
    detalhes NVARCHAR(MAX),  
    acessado_em DATETIME2 NOT NULL DEFAULT SYSDATETIME(),  
    CONSTRAINT chk_resultado  
        CHECK (resultado IN ('sucesso', 'negado', 'erro'))  
);  
GO
```

```
/* =====  
REGISTRO DESCARTE LGPD  
===== */  
CREATE TABLE registro_descarte (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    tabela_origem VARCHAR(100) NOT NULL,  
    criterio VARCHAR(300) NOT NULL,  
    qtd_registros INT NOT NULL,  
    autorizado_por VARCHAR(150) NOT NULL,  
    area_autorizadora VARCHAR(30) NOT NULL,  
    metodo_descarte VARCHAR(30) NOT NULL,  
    descartado_em DATETIME2 NOT NULL DEFAULT SYSDATETIME(),  
    observacao VARCHAR(MAX),  
    CONSTRAINT chk_qtd CHECK (qtd_registros >= 0)  
);  
GO  
  
/* =====  
CLIENTES HISTÓRICO (LGPD)  
===== */  
CREATE TABLE clientes_historico (  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    cliente_id_orig INT NOT NULL,  
    nome_anonimizado VARCHAR(50) NOT NULL,  
    inativado_em DATE NULL,  
    arquivado_em DATETIME2 NOT NULL DEFAULT SYSDATETIME()  
);  
GO
```

```
/* =====
ÍNDICES
===== */
CREATE INDEX idx_logs_data ON logs_acesso(acessado_em);
CREATE INDEX idx Leituras_data ON leituras_sensores(coletado_em);
CREATE INDEX idx_nf_cliente ON notas_fiscais(cliente_id);
GO

/* =====
TRIGGER LOG CLIENTES
===== */
CREATE TRIGGER trg_clientes_insert
ON clientes
AFTER INSERT
AS
BEGIN
    INSERT INTO logs_acesso (usuario, acao, tabela_afetada, registro_id, resultado, detalhes)
    SELECT SYSTEM_USER, 'INSERT', 'clientes', id, 'sucesso', nome
    FROM inserted;
END;
GO

CREATE TRIGGER trg_clientes_delete
ON clientes
AFTER DELETE
AS
BEGIN
    INSERT INTO logs_acesso (usuario, acao, tabela_afetada, registro_id, resultado, detalhes)
    SELECT SYSTEM_USER, 'DELETE', 'clientes', id, 'sucesso', nome
    FROM deleted;
END;
GO

/* =====
DADOS DE TESTE
===== */
INSERT INTO clientes (nome, cpf_cnpj, email, ativo) VALUES
('Empresa A', '1111111111', 'a@email.com', 1),
('Empresa B', '2222222222', 'b@email.com', 1);

INSERT INTO equipamentos (tag, descricao, tipo, instalado_em) VALUES
('TI-101', 'Sensor temperatura', 'sensor', '2023-01-01'),
('FT-202', 'Medidor fluxo', 'medidor', '2023-02-01');

INSERT INTO leituras_sensores (equipamento_id, valor, unidade, qualidade)
VALUES (1, 25.5, '°C', 'bom');

INSERT INTO notas_fiscais (cliente_id, numero_nf, valor_total, emitida_em)
VALUES (1, 'NF001', 1000, GETDATE());
GO
willdb.sql
Exibindo willdb.sql.
```

7. Matriz de Riscos

Risco	Nível	Medida Preventiva
Guardar dados eternamente	● Alto	Implementar política de retenção + eventos agendados
Apagar dados cedo demais	● Alto	Validar prazo legal antes de qualquer exclusão
Descarte sem autorização	● Alto	Toda exclusão são permanentes
Dados pessoais sem anonimizar	● Alto	Executar sp_anonimizar_clientes_inativos mensalmente
Sem log de auditoria	● Médio	Triggers automáticos em tabelas críticas
Backup sem verificação	● Médio	Registrar resultado em tabela backups; verificar diário
Dados de teste em produção	● Médio	Separar ambientes; nunca importar carga de teste real

8. Estrutura do Projeto MySQL

8.1 Tabelas Implementadas

- clientes — Cadastro de clientes com controle de atividade e data de inativação
- equipamentos — Instrumentos e sensores da planta industrial
- leituras_sensores — Dados coletados pelos sensores IoT em tempo real
- notas_fiscais — Documentos fiscais vinculados a clientes
- backups — Registro de execuções de backup
- logs_acesso — Auditoria de todos os acessos e operações
- registro_descarte — Comprovante formal de cada descarte realizado
- clientes_historico — Versão anonimizada de clientes descartados (LGPD)

8.2 Stored Procedures

- sp_arquivar_leituras_antigas(meses, usuario, OUT qtd) — Arquivo leituras antigas
- sp_anonimizar_clientes_inativos(anos, usuario, OUT qtd) — LGPD: anonimiza clientes
- sp_limpar_logs_antigos(meses, usuario, OUT qtd) — Remove logs expirados

8.3 Eventos Agendados

- evt_arquivar_leituras — Diário às 02:00: arquiva leituras > 12 meses
- evt_anonimizar_clientes — Mensal (dia 1 às 03:00): anonimiza clientes inativos > 2 anos

8.4 Triggers de Auditoria

- `trg_audit_nf_delete` — Registra qualquer DELETE em `notas_fiscais`
- `trg_audit_clientes_update` — Registra alterações em campos críticos de clientes

8.5 Views Operacionais

- `vw_clientes_para_descarte` — Clientes inativos > 2 anos candidatos à anonimização
- `vw_leituras_para_arquivo` — Leituras > 12 meses candidatas ao arquivo frio
- `vw_tamanho_tabelas` — Visão de tamanho e crescimento de cada tabela

9. Conclusão

A administração eficaz de um banco de dados vai muito além de criar tabelas e realizar consultas SQL. Envolve um conjunto estruturado de políticas, procedimentos e automações que garantem:

- Conformidade legal (LGPD, legislação fiscal, normas setoriais)
- Segurança e rastreabilidade através de auditoria contínua
- Performance sustentável ao longo do ciclo de vida do sistema
- Governança clara sobre quem pode fazer o quê com os dados
- Redução de custos ao eliminar armazenamento desnecessário

O projeto implementado neste manual demonstra como esses conceitos se traduzem em código MySQL real, com stored procedures, triggers, eventos agendados e views que automatizam a gestão do ciclo de vida dos dados em um ambiente de automação industrial.